

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

д-р техн. наук, профессор
должность, уч. степень, звание

подпись, дата

А.В. Бржезовский
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №6

ХРАНИМЫЕ ПРОЦЕДУРЫ И
ФУНКЦИИ

по курсу: Методы и средства проектирования
информационных систем и технологий

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

И.А. Лаврешин
инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Изучить хранимые процедуры и функции в PostgreSQL: вставка с пополнением справочника, удаление с проверками и ручным каскадом, возврат агрегата, статистика во временной таблице, блок с управляющими конструкциями, скалярная и табличная функции.

2 Задание и исходные данные

Вставку с пополнением справочников (например, вставляется информация о студенте, если указанный номер группы отсутствует в БД, запись добавляется в таблицу с перечнем групп).

Удаление с очисткой справочников (удаляется информация о студенте, если в его группе нет больше студентов, запись удаляется из таблицы с перечнем групп).

Каскадное удаление (при наличии условия ссылочной целостности по action перед удалением записи о группе удаляются записи обо всех студентах этой группы).

Вычисление и возврат значения агрегатной функции (на примере одного из запросов из задания).

Формирование статистики во временной таблице (например, для рассматриваемой БД).

Для каждого факультета: количество групп, количество обучающихся студентов, количество изучаемых дисциплин, средний балл по факультету).

Самостоятельно предложить и реализовать ПЗ или ХП, демонстрирующие использование конструкций, описанных в п. 6.1.

Самостоятельно предложить и реализовать запросы, использующие скалярную и табличную функцию по аналогии с примерами в п. 6.3.

3 Ход выполнения задания

Создана процедура lr6_insert_hire_with_position: перед вставкой строки в таблицу “прием” проверяется наличие сотрудника и подразделения: если в справочнике “должность” нет строки с переданным идентификатором, она добавляется (наименование формируется служебной строкой с указанием ид). Затем вычисляется следующий свободный ид приёма и выполняется INSERT. Результаты представлены на рисунках 1 – 2.

```
1 CREATE OR REPLACE PROCEDURE lr6_insert_hire_with_position(  
2     IN p_сотрудник int,  
3     IN p_подразделение int,  
4     IN p_должность int,  
5     IN p_дата date,  
6     IN p_совместительство bit(1)  
7 )  
8 LANGUAGE plpgsql  
9 AS $$  
10 DECLARE  
11     v_next int;  
12 BEGIN  
13     IF NOT EXISTS (SELECT 1 FROM сотрудник WHERE ид = p_сотрудник) THEN  
14         RAISE EXCEPTION 'Сотрудник % не найден', p_сотрудник;  
15     END IF;  
16     IF NOT EXISTS (SELECT 1 FROM подразделение WHERE ид = p_подразделение) THEN  
17         RAISE EXCEPTION 'Подразделение % не найдено', p_подразделение;  
18     END IF;  
19     IF NOT EXISTS (SELECT 1 FROM должность WHERE ид = p_должность) THEN  
20         INSERT INTO должность (ид, наименование, категория)  
21             VALUES (  
22                 p_должность,  
23                 'LR6 должность id ' || p_должность::text,  
24                 'прочее'  
25             );  
26     END IF;  
27     SELECT COALESCE(MAX(ид), 0) + 1 INTO v_next FROM прием;  
28  
29     INSERT INTO прием (ид, ид_сотрудника, ид_подразделения, ид_должности, дата_приема, совместительство)  
30     VALUES (v_next, p_сотрудник, p_подразделение, p_должность, p_дата, p_совместительство);  
31 END;  
32 $$;
```

Рисунок 1 – Процедура “lr6_insert_hire_with_position”

	раздел text	ид integer	наименование_или_фio character varying	категория character varying	ид_сотрудника integer	ид_подразделения integer	ид_должности integer	дата_приема date	совместительство text
1	должность (справочн...	90	LR6 должность id 90	прочее	[null]	[null]	[null]	[null]	[null]
2	прием (новая строка)	13	Иванов Иван Иванович	[null]	1	2	90	2026-05-19	0

Рисунок 2 – Результаты работы программы

Далее процедура lr6_delete_employee вручную удаляет зависимые строки в “перевод”, “увольнение”, “прием”, “дети” и после этого удаляет сотрудника – это демонстрация “каскада” при политике NO ACTION на внешних ключах. Результаты представлены на рисунках 3 – 4.

```

CREATE OR REPLACE PROCEDURE lr6_delete_employee(IN p_ид int)
LANGUAGE plpgsql
AS $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM сотрудник WHERE ид = p_ид) THEN
        RAISE EXCEPTION 'Сотрудник % не найден', p_ид;
    END IF;

    DELETE FROM перевод WHERE ид_сотрудника = p_ид;
    DELETE FROM увольнение WHERE ид_сотрудника = p_ид;
    DELETE FROM прием WHERE ид_сотрудника = p_ид;
    DELETE FROM дети WHERE ид_сотрудника = p_ид;
    DELETE FROM сотрудник WHERE ид = p_ид;
END;

```

Рисунок 3 – Процедура “lr6_delete_employee”

	ид [PK] integer	фio character varying (200)
1	8	Васильев Тимур Тимурович

Рисунок 4 – Результаты работы программы

После чего процедура lr6_delete_leaf_subdept удаляет подразделение только если оно “листовое”: нет дочерних записей в “подразделение”, нет строк штатного расписания и нет приёмов на это подразделение. При нарушении условий вызывается RAISE EXCEPTION с пояснением. Результаты представлены на рисунке 5.

```

1 CREATE OR REPLACE PROCEDURE lr6_delete_leaf_subdept(IN p_ид int)
2 LANGUAGE plpgsql
3 AS $$
4 BEGIN
5     IF NOT EXISTS (SELECT 1 FROM подразделение WHERE ид = p_ид) THEN
6         RAISE EXCEPTION 'Подразделение % не найдено', p_ид;
7     END IF;
8
9     IF EXISTS (SELECT 1 FROM подразделение WHERE ид_родительское = p_ид) THEN
10        RAISE EXCEPTION 'Подразделение %: есть дочерние записи', p_ид;
11    END IF;
12
13    IF EXISTS (SELECT 1 FROM штатное_расписание WHERE ид_подразделения = p_ид) THEN
14        RAISE EXCEPTION 'Подразделение %: есть штатное расписание', p_ид;
15    END IF;
16
17    IF EXISTS (SELECT 1 FROM прием WHERE ид_подразделения = p_ид) THEN
18        RAISE EXCEPTION 'Подразделение %: есть приёмы', p_ид;
19    END IF;
20
21    DELETE FROM подразделение WHERE ид = p_ид;
22 END;
23 $$;

```

Рисунок 5 – Процедура “lr6_delete_leaf_subdept”

Далее функция lr6_sum_staff_units возвращает сумму поля “количество_единиц” по заданному подразделению из таблицы

“штатное_расписание” (тип возврата bigint). Результаты представлены на рисунках 6 – 7.

```
CREATE OR REPLACE FUNCTION lr6_sum_staff_units(p_подразделение int)
RETURNS bigint
LANGUAGE sql
STABLE
AS $$
    SELECT COALESCE(SUM(количество_единиц), 0)::bigint
    FROM штатное_расписание
    WHERE ид_подразделения = p_подразделение;
$$;
```

Рисунок 6 – Процедура “lr6_sum_staff_units”

	ид [PK] integer	наименование character varying (200)	сумма_штатных_единиц bigint
1	1	ООО «Промтех»	0
2	2	Цех механической обработ...	8
3	3	Отдел главного конструктор...	3
4	4	Сектор расчётов ОГК	3

Рисунок 7 – Результаты работы программы

Процедура lr6_fill_stats_temp создаёт временную таблицу lr6_stat и заполняет её по каждому подразделению: число строк штата, сумму штатных единиц (через вызов lr6_sum_staff_units) и число различных сотрудников по таблице “прием”. Результаты представлены на рисунках 8 и 9.

```
1 CREATE OR REPLACE PROCEDURE lr6_fill_stats_temp()
2 LANGUAGE plpgsql
3 AS $$
4 BEGIN
5     DROP TABLE IF EXISTS lr6_stat;
6     CREATE TEMP TABLE lr6_stat (
7         ид_подразделения int PRIMARY KEY,
8         наименование varchar(200) NOT NULL,
9         строк_штат int NOT NULL,
10        сумма_штатных_единиц bigint NOT NULL,
11        работников_по_приему bigint NOT NULL
12    );
13
14    INSERT INTO lr6_stat (ид_подразделения, наименование, строк_штат, сумма_штатных_единиц, работников_по_приему)
15    SELECT
16        пд.ид,
17        пд.наименование,
18        (SELECT COUNT(*)::int FROM штатное_расписание ш WHERE ш.ид_подразделения = пд.ид),
19        lr6_sum_staff_units(пд.ид),
20        (SELECT COUNT(DISTINCT пр.ид_сотрудника)::bigint FROM прием пр WHERE пр.ид_подразделения = пд.ид)
21    FROM подразделение пд;
22 END;
```

Рисунок 8 – Процедура “lr6_fill_stats_temp”

	ид_подразделения [PK] integer	наименование character varying (200)	строк_штат integer	сумма_штатных_единиц bigint	работников_по_приему bigint
1	1	ООО «Промтех»	0	0	0
2	2	Цех механической обработ...	2	8	4
3	3	Отдел главного конструктора...	2	3	7
4	4	Сектор расчётов ОГК	2	3	1

Рисунок 9 – Результаты работы программы

Файл 06_batch_control_flow содержит анонимный блок DO на языке plpgsql: цикл FOR по подразделениям с проверкой IF EXISTS (приёмы), счётчик подразделений с приёмами, затем цикл WHILE с выводом NOTICE — демонстрация п. 6.1. Результаты представлены на рисунках 10 – 11.

```

1 DROP TABLE IF EXISTS _lr6_script_result;
2 CREATE TEMP TABLE _lr6_script_result (
3     шаг text NOT NULL,
4     значение text NOT NULL
5 );
6
7 DO $$
8 DECLARE
9     v_ид int;
10    v_cnt int := 0;
11    v_step int;
12 BEGIN
13     FOR v_ид IN SELECT пд.ид FROM подразделение пд ORDER BY пд.ид
14     LOOP
15         IF EXISTS (SELECT 1 FROM прием пр WHERE пр.ид_подразделения = v_ид) THEN
16             v_cnt := v_cnt + 1;
17         END IF;
18     END LOOP;
19
20     INSERT INTO _lr6_script_result
21     VALUES ('подразделений с хотя бы одним приёмом', v_cnt::text);
22
23     v_step := 3;
24     WHILE v_step > 0 LOOP
25         INSERT INTO _lr6_script_result
26         VALUES ('while step ' || v_step::text, v_step::text);
27         v_step := v_step - 1;
28     END LOOP;
29 END;
30 $$;

```

Рисунок 10 – Процедура “06_batch_control_flow”

	шаг text	значение text
1	while step 1	1
2	while step 2	2
3	while step 3	3
4	подразделений с хотя бы одним приёмом	3

Рисунок 11 – Результаты работы программы

Функция `lr6_child_count` возвращает число строк в «дети» для сотрудника. Функция `lr6_direct_subdepts` возвращает набор строк (ид и наименование) непосредственно подчинённых подразделений для заданного родителя (или корневые, если параметр `NULL`). Результаты представлены на рисунках 12 и 13.

```
CREATE OR REPLACE FUNCTION lr6_child_count(p_сотрудник int)
RETURNS int
LANGUAGE sql
STABLE
AS $$
    SELECT COUNT(*)::int FROM дети WHERE ид_сотрудника = p_сотрудник;
$$;
```

Рисунок 12 – Процедура “`lr6_child_count`”

	ид [PK] integer	фио character varying (200)	детей integer
1	1	Иванов Иван Иванович	2
2	2	Петрова Мария Сергеевна	1
3	3	Сидоров Пётр Алексеевич	1
4	4	Козлова Анна Викторовна	2
5	5	Николаев Олег Олегович	0
6	6	Большая Семья Инженер	7
7	7	Только техники (руководитель це...	0
8	8	Васильев Тимур Тимурович	0
9	9	Техник ОГК А	0
10	10	Техник ОГК Б	0
11	11	Техник ОГК В	0

Рисунок 13 – Результаты работы программы

Далее функция `lr6_direct_subdepts` – табличная: `RETURNS TABLE` (ид, наименование). В теле один `SELECT` с условием на иерархию: если параметр `p_родитель NULL`, возвращаются только корневые подразделения (где `ид_родительское IS NULL`), если не `NULL` – все строки, у которых `ид_родительское` равен переданному родителю. Так получают “один уровень” дерева без рекурсии. Результаты представлены на рисунках 14 – 15.


```
CREATE OR REPLACE FUNCTION lr6_direct_subdepts(p_родитель int)
RETURNS TABLE(ид int, наименование varchar(200))
LANGUAGE sql
STABLE
AS $$
    SELECT п.ид, п.наименование
    FROM подразделение п
    WHERE (p_родитель IS NULL AND п.ид_родительское IS NULL)
        OR (p_родитель IS NOT NULL AND п.ид_родительское = p_родитель);
$$;
```

Рисунок 14 – Процедура “lr6_direct_subdepts”

	выборка text	ид integer	наименование character varying
1	дочерние у ид=1 (Промтех)	2	Цех механической обработки
2	дочерние у ид=1 (Промтех)	3	Отдел главного конструктора

Рисунок 15 – Результаты работы программы

4 Вывод

Процедуры позволяют инкапсулировать сложную последовательность DML и проверок; временная таблица `lr6_stat` существует только в рамках сессии и удобна для промежуточной отчётности. Табличная функция упрощает использование результата в FROM без временной таблицы вручную.

ПРИЛОЖЕНИЕ А

Листинг с кодом программы

```
-- ===== --
-- 01_proc_insert_with_directory.sql
-- ===== --

CREATE OR REPLACE PROCEDURE lr6_insert_hire_with_position(
    IN p_сотрудник int,
    IN p_подразделение int,
    IN p_должность int,
    IN p_дата date,
    IN p_совместительство bit(1)
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_next int;
BEGIN
    IF NOT EXISTS (SELECT 1 FROM сотрудник WHERE ид = p_сотрудник)
    THEN
        RAISE EXCEPTION 'Сотрудник % не найден', p_сотрудник;
    END IF;

    IF NOT EXISTS (SELECT 1 FROM подразделение WHERE ид =
p_подразделение) THEN
        RAISE EXCEPTION 'Подразделение % не найдено', p_подразделение;
    END IF;

    IF NOT EXISTS (SELECT 1 FROM должность WHERE ид = p_должность)
    THEN
        INSERT INTO должность (ид, наименование, категория)
        VALUES (
```

```

        p_должность,
        'LR6 должность id ' || p_должность::text,
        'прочее'
    );
END IF;

```

```

SELECT COALESCE(MAX(ид), 0) + 1 INTO v_next FROM прием;

```

```

INSERT INTO прием (ид, ид_сотрудника, ид_подразделения, ид_должности,
дата_приема, совместительство)
VALUES (v_next, p_сотрудник, p_подразделение, p_должность, p_дата,
p_совместительство);
END;
$$;

```

```

-- =====
-- 02_proc_delete_employee_cascade.sql
-- =====

```

```

CREATE OR REPLACE PROCEDURE lr6_delete_employee(IN p_ид int)
LANGUAGE plpgsql
AS $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM сотрудник WHERE ид = p_ид) THEN
        RAISE EXCEPTION 'Сотрудник % не найден', p_ид;
    END IF;

    DELETE FROM перевод WHERE ид_сотрудника = p_ид;
    DELETE FROM увольнение WHERE ид_сотрудника = p_ид;
    DELETE FROM прием WHERE ид_сотрудника = p_ид;
    DELETE FROM дети WHERE ид_сотрудника = p_ид;

```

```

DELETE FROM сотрудник WHERE ид = p_ид;
END;
$$;

-- =====
-- 03_proc_delete_subdivision_safe_reset.sql
-- =====

CREATE OR REPLACE PROCEDURE lr6_delete_leaf_subdept(IN p_ид int)
LANGUAGE plpgsql
AS $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM подразделение WHERE ид = p_ид) THEN
        RAISE EXCEPTION 'Подразделение % не найдено', p_ид;
    END IF;

    IF EXISTS (SELECT 1 FROM подразделение WHERE ид_родительское =
p_ид) THEN
        RAISE EXCEPTION 'Подразделение %: есть дочерние записи', p_ид;
    END IF;

    IF EXISTS (SELECT 1 FROM штатное_расписание WHERE
ид_подразделения = p_ид) THEN
        RAISE EXCEPTION 'Подразделение %: есть штатное расписание', p_ид;
    END IF;

    IF EXISTS (SELECT 1 FROM прием WHERE ид_подразделения = p_ид)
THEN
        RAISE EXCEPTION 'Подразделение %: есть приёмы', p_ид;
    END IF;

```

```

DELETE FROM подразделение WHERE ид = p_ид;
END;
$$;

-- =====
-- 04_fn_aggregate.sql
-- =====

CREATE OR REPLACE FUNCTION lr6_sum_staff_units(p_подразделение int)
RETURNS bigint
LANGUAGE sql
STABLE
AS $$
    SELECT COALESCE(SUM(количество_единиц), 0)::bigint
    FROM штатное_расписание
    WHERE ид_подразделения = p_подразделение;
$$;

-- =====
-- 05_proc_stats_temp.sql
-- =====

CREATE OR REPLACE PROCEDURE lr6_fill_stats_temp()
LANGUAGE plpgsql
AS $$
BEGIN
    DROP TABLE IF EXISTS lr6_stat;
    CREATE TEMP TABLE lr6_stat (
        ид_подразделения int PRIMARY KEY,
        наименование varchar(200) NOT NULL,
        строк_штат int NOT NULL,
        сумма_штатных_единиц bigint NOT NULL,

```

```

        работников_по_приему bigint NOT NULL
    );

    INSERT INTO lr6_stat (ид_подразделения, наименование, строк_штат,
сумма_штатных_единиц, работников_по_приему)
    SELECT
        пд.ид,
        пд.наименование,
        (SELECT COUNT(*)::int FROM штатное_расписание ш WHERE
ш.ид_подразделения = пд.ид),
        lr6_sum_staff_units(пд.ид),
        (SELECT COUNT(DISTINCT пр.ид_сотрудника)::bigint FROM прием пр
WHERE пр.ид_подразделения = пд.ид)
    FROM подразделение пд;
END;
$$;

-- =====
-- 06_batch_control_flow.sql
-- =====

DROP TABLE IF EXISTS _lr6_script_result;
CREATE TEMP TABLE _lr6_script_result (
    шаг text NOT NULL,
    значение text NOT NULL
);
DO $$
DECLARE
    v_ид int;
    v_cnt int := 0;
    v_step int;

```

```

BEGIN
    FOR v_ид IN SELECT пд.ид FROM подразделение пд ORDER BY пд.ид
    LOOP
        IF EXISTS (SELECT 1 FROM прием пр WHERE пр.ид_подразделения =
v_ид) THEN
            v_cnt := v_cnt + 1;
        END IF;
    END LOOP;

    INSERT INTO _lr6_script_result
    VALUES ('подразделений с хотя бы одним приёмом', v_cnt::text);

    v_step := 3;
    WHILE v_step > 0 LOOP
        INSERT INTO _lr6_script_result
        VALUES ('while step ' || v_step::text, v_step::text);
        v_step := v_step - 1;
    END LOOP;
END;
$$;

-- =====
-- 07_fn_scalar.sql
-- =====

CREATE OR REPLACE FUNCTION lr6_child_count(p_сотрудник int)
RETURNS int
LANGUAGE sql
STABLE
AS $$

    SELECT COUNT(*)::int FROM дети WHERE ид_сотрудника = p_сотрудник;

```


\$\$;

-- ===== --

-- 08_fn_table.sql

-- ===== --

CREATE OR REPLACE FUNCTION lr6_direct_subdepts(p_родитель int)

RETURNS TABLE(ид int, наименование varchar(200))

LANGUAGE sql

STABLE

AS \$\$

SELECT п.ид, п.наименование

FROM подразделение п

WHERE (p_родитель IS NULL AND п.ид_родительское IS NULL)

OR (p_родитель IS NOT NULL AND п.ид_родительское = p_родитель);

\$\$;